

Project 3

2025-11-17

Shivsagar did Question 1. Advaiith Pathi did question 2. Rayan did question 3. Akshay did question 4. Saaqib did question 5. We discussed all the questions and decided to split the work. We helped each other with the questions.

Question 1

```
# Load packages
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    4.0.0      ✓ tibble     3.3.0
## ✓ lubridate  1.9.4      ✓ tidyr      1.3.1
## ✓ purrr      1.1.0
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rstanarm)
```

```
## Loading required package: Rcpp
## This is rstanarm version 2.32.2
## - See https://mc-stan.org/rstanarm/articles/priors for changes to default priors!
## - Default priors may change, so it's safest to specify priors, even if equivalent to the defaults.
## - For execution on a local, multicore CPU with excess RAM we recommend calling
##   options(mc.cores = parallel::detectCores())
```

```
library(rstan)
```

```
## Loading required package: StanHeaders
##
## rstan version 2.32.7 (Stan version 2.32.2)
##
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
## change `threads_per_chain` option:
## rstan_options(threads_per_chain = 1)
##
##
## Attaching package: 'rstan'
##
## The following object is masked from 'package:tidyr':
##
##      extract
```

```
rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())

knitr::opts_chunk$set(echo = TRUE)

hibbs <- read.table("hibbs.dat", header = TRUE)

fit_stan <- stan_glm(vote ~ growth,
                    data    = hibbs,
                    family  = gaussian(),
                    refresh = 0)
# Get all posterior draws as a matrix
draws <- as.matrix(fit_stan)

colnames(draws)
```

```
## [1] "(Intercept)" "growth"      "sigma"
```

```
# Posterior mean of sigma
sigma_stan <- mean(draws[, "sigma"])
sigma_stan
```

```
## [1] 4.007752
```

```
fit_lm <- lm(vote ~ growth, data = hibbs)

resid_lm <- resid(fit_lm)
sigma_MLE <- sqrt(mean(resid_lm^2))
sigma_MLE
```

```
## [1] 3.520233
```

```
# equivalently: summary(fit_lm)$sigma

# Leave-one-out cross validation for vote

n <- nrow(hibbs)
r_cv <- numeric(n)

for (i in 1:n) {
  # fit model to all points except i
  fit_minus_i <- lm(vote ~ growth, data = hibbs[-i, ])

  # predict the held-out point i
  yhat_i <- predict(fit_minus_i, newdata = hibbs[i, , drop = FALSE])

  # cross-validated residual for point i
  r_cv[i] <- hibbs$vote[i] - yhat_i
}

sigma_CV <- sqrt(mean(r_cv^2))
sigma_CV
```

```
## [1] 3.906604
```

#Across the three different approaches, the estimates are similar but not identical, and each one reflects a slightly different idea of “error size”. #i) The Posterior Mean (4.04): This is the biggest estimate out of the three. Stan uses a Bayesian model with #priors on the parameters and integrates over the whole posterior distribution. Because the data set is #small, the Bayesian model tends to be a lot more conservative and allows for kind of a higher uncertainty, #pushing the value higher. #ii) The MLE Residual RMS from lm (3.52): This is the smallest estimate. It is just the square root of the #average squared residuals from the fitted model. MLE usually underestimates uncertainty a little because #it evaluates the model on the same data used to fit it. It doesn’t dock points for over fitting #iii) The Cross-validated Value (3.91): This value sits in the middle as

LOOCV measures prediction error on #points not really used for fitting. As a result, the model cannot over fit each observation. Because of that, the value is usually larger than the MLE estimate, but still slightly lower than the Bayesian estimate.

Question 2

```
# Create a fake dataset with a big difference between sigma_CV and sigma_MLE

set.seed(123)

# Strategy: Create a dataset with one or more high-leverage outliers
# High-leverage points are far from the mean of x and strongly influence the fitted line
# When these points are removed in LOOCV, the fitted line changes dramatically

# Create base data: most points follow a clear linear pattern
n <- 20
x <- c(seq(0, 10, length.out = n-1), 50) # Add one extreme x value
y <- c(2 + 3 * seq(0, 10, length.out = n-1) + rnorm(n-1, 0, 1),
      2 + 3 * 50 + 20) # Last point follows trend but with large deviation

# Alternative: Create influential outlier that pulls the line
# x <- c(seq(0, 10, length.out = n-1), 50)
# y <- c(2 + 3 * seq(0, 10, length.out = n-1) + rnorm(n-1, 0, 1), 50)

data <- data.frame(x = x, y = y)

# Fit full model
fit_full <- lm(y ~ x, data = data)

# (i) MLE estimate
residuals_full <- residuals(fit_full)
sigma_MLE <- sqrt(sum(residuals_full^2) / n)

# (ii) Leave-one-out CV estimate
cv_residuals <- numeric(n)

for (i in 1:n) {
  # Fit model excluding observation i
  fit_minus_i <- lm(y ~ x, data = data[-i, ])

  # Predict held-out observation
  pred_i <- predict(fit_minus_i, newdata = data[i, ])

  # CV residual
  cv_residuals[i] <- data$y[i] - pred_i
}
```

```
sigma_CV <- sqrt(sum(cv_residuals^2) / n)

# Display results

cat("sigma_MLE: ", round(sigma_MLE, 4), "\n")
```

```
## sigma_MLE:  1.5952
```

```
cat("sigma_CV:  ", round(sigma_CV, 4), "\n")
```

```
## sigma_CV:   4.8647
```

```
cat("Difference: ", round(sigma_CV - sigma_MLE, 4), "\n")
```

```
## Difference:  3.2695
```

```
cat("Ratio (CV/MLE): ", round(sigma_CV / sigma_MLE, 4), "\n\n")
```

```
## Ratio (CV/MLE):  3.0496
```

```
# Show the outlier's impact
cat("Impact of high-leverage point:\n")
```

```
## Impact of high-leverage point:
```

```
cat("=====\n")
```

```
## =====
```

```
cat("Point", n, ": x =", x[n], ", y =", y[n], "\n")
```

```
## Point 20 : x = 50 , y = 172
```

```
cat("Full model residual for this point:", round(residuals_full[n], 4), "\n")
```

```
## Full model residual for this point: 1.6293
```

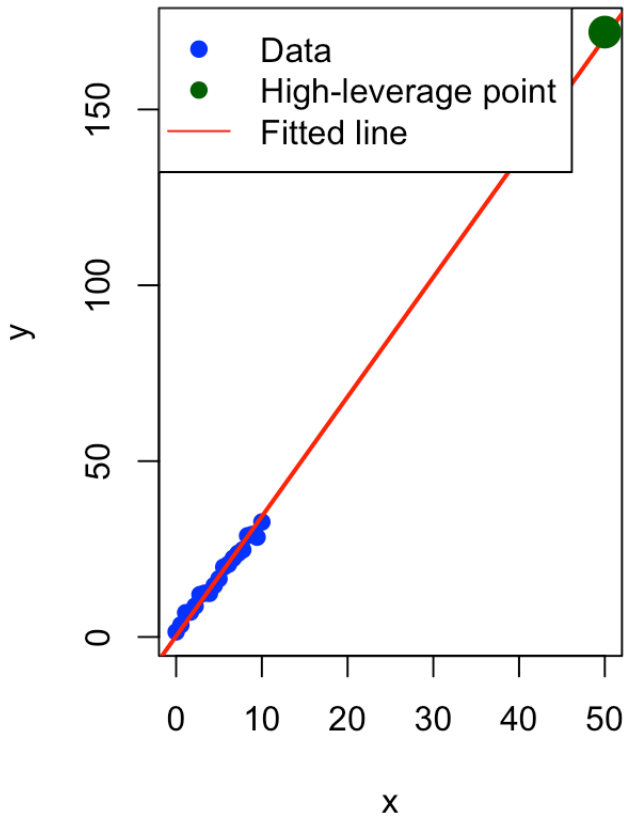
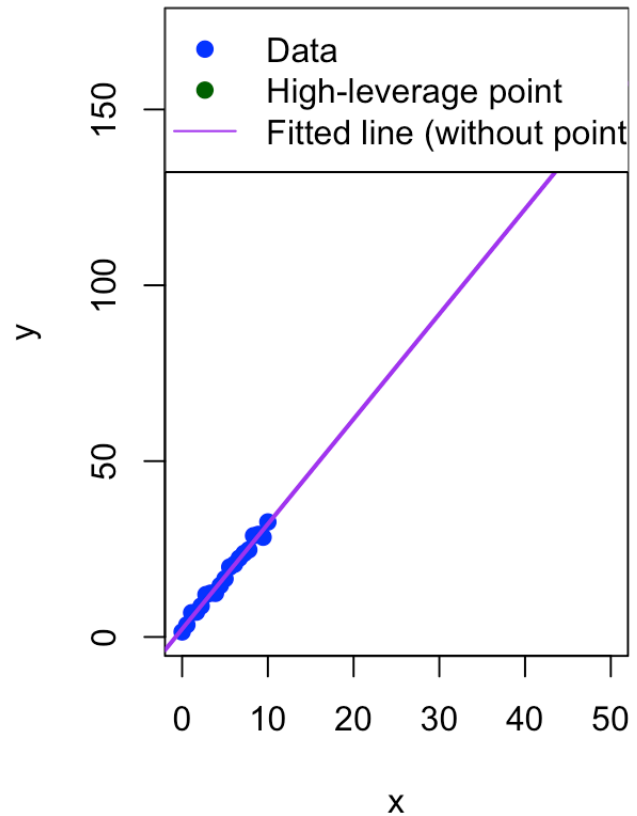
```
cat("CV residual for this point:", round(cv_residuals[n], 4), "\n\n")
```

```
## CV residual for this point: 20.4691
```

```
# Plot the data
par(mfrow = c(1, 2))

# Plot 1: Full model
plot(x, y, main = "Full Model Fit",
     xlab = "x", ylab = "y", pch = 19, col = "blue")
abline(fit_full, col = "red", lwd = 2)
points(x[n], y[n], col = "darkgreen", pch = 19, cex = 2)
legend("topleft", legend = c("Data", "High-leverage point", "Fitted line"),
     col = c("blue", "darkgreen", "red"), pch = c(19, 19, NA), lty = c(NA, NA, 1))

# Plot 2: Model without the high-leverage point
fit_without <- lm(y ~ x, data = data[-n, ])
plot(x, y, main = "Fit Without High-Leverage Point",
     xlab = "x", ylab = "y", pch = 19, col = "blue")
abline(fit_without, col = "purple", lwd = 2)
points(x[n], y[n], col = "darkgreen", pch = 19, cex = 2)
legend("topleft", legend = c("Data", "High-leverage point", "Fitted line (without point)"),
     col = c("blue", "darkgreen", "purple"), pch = c(19, 19, NA), lty = c(NA, NA, 1))
```

Full Model Fit**Fit Without High-Leverage Point**

#I created a small dataset with 20 points to show how a high-leverage outlier can affect regression. In the data, 19 points follow a tight straight-line pattern with x-values between 0 and 10, while one point has an extreme x-value of 50, making it a high-leverage point. When we fit a line to all 20 points, this outlier pulls the line toward itself, and because the line passes so close to this extreme point, its residual is very small, just 1.63. As a result, the usual estimate of the standard error, $\sigma_{MLE} = 1.5952$ looks tiny and underestimates how uncertain predictions actually are. However, when we do leave-one-out cross-validation (LOOCV), we remove the outlier and fit the model to the remaining 19 points. The fitted line now reflects the pattern of the main group, and when we use this model to predict the outlier at $x = 50$, the prediction is very far off, giving a huge residual of 20.47. This demonstrates the “masking effect” of high-leverage points: they pull the regression line toward themselves, making their own residuals look small and giving a misleadingly low estimate of prediction error. Cross-validation reveals this by showing that if we had fit the model without the outlier, we would predict it very poorly, giving $\sigma_{CV} = 4.8647$ which is a more honest measure of out-of-sample prediction error.

Question 3

```

library(tidyverse)
library(nflfastR)
pbp <- load_pbp(2023)

# Load 2023 play-by-play data
fourth_down <- pbp |>
  filter(down == 4, !is.na(ydstogo), !is.na(yardline_100)) |>
  mutate(
    go_for_it = as.integer(play_type %in% c("run", "pass")),
    posteam_type = if_else(posteam == home_team, "home", "away"),
    score_differential = posteam_score - defteam_score
  ) |>
  select(go_for_it, yardline_100, ydstogo, score_differential,
         posteam_type, epa, wpa) |>
  drop_na()

##The code below fits a linear regression for EPA ~ go_for_it * yardline_100 + contro
ls
m_epa <- lm(epa ~ go_for_it * yardline_100 + ydstogo + score_differential + posteam_t
ype,
            data = fourth_down)

summary(m_epa)$coefficients

```

##	Estimate	Std. Error	t value	Pr(> t)
## (Intercept)	0.067328801	0.076329703	0.8820786	0.37778165
## go_for_it	0.050361357	0.135600948	0.3713938	0.71036179
## yardline_100	-0.001337055	0.001143771	-1.1689891	0.24247027
## ydstogo	-0.005451740	0.004660985	-1.1696541	0.24220248
## score_differential	0.001059333	0.002459179	0.4307670	0.66665857
## posteam_typehome	0.103309404	0.050049816	2.0641315	0.03906274
## go_for_it:yardline_100	-0.002204985	0.002870969	-0.7680281	0.44251099

```
confint(m_epa)
```

##	2.5 %	97.5 %
## (Intercept)	-0.082315133	0.2169727341
## go_for_it	-0.215483505	0.3162062200
## yardline_100	-0.003579411	0.0009053001
## ydstogo	-0.014589574	0.0036860935
## score_differential	-0.003761872	0.0058805383
## posteam_typehome	0.005187034	0.2014317747
## go_for_it:yardline_100	-0.007833503	0.0034235331

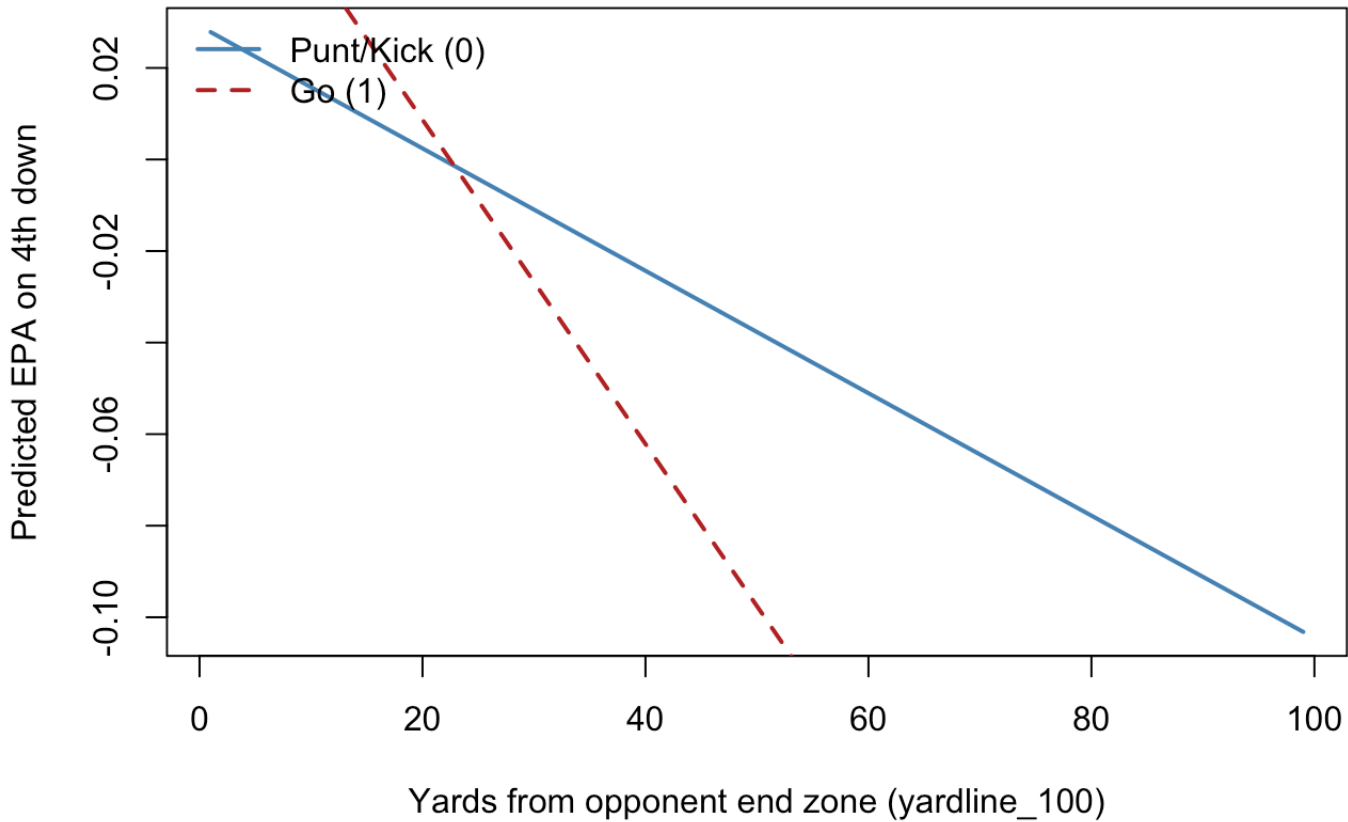

```
xg <- seq(min(fourth_down$yardline_100, na.rm=TRUE),
          max(fourth_down$yardline_100, na.rm=TRUE),
          length.out = 200)

med_yds <- median(fourth_down$ydstogo, na.rm=TRUE)
med_sd <- median(fourth_down$score_differential, na.rm=TRUE)
mode_pt <- names(which.max(table(fourth_down$posteam_type)))

pred_punt <- predict(m_epa, newdata = data.frame(
  epa = NA, go_for_it = 0, yardline_100 = xg,
  ydstogo = med_yds, score_differential = med_sd,
  posteam_type = mode_pt
))
pred_go <- predict(m_epa, newdata = data.frame(
  epa = NA, go_for_it = 1, yardline_100 = xg,
  ydstogo = med_yds, score_differential = med_sd,
  posteam_type = mode_pt
))

plot(xg, pred_punt, type = "l", lwd = 2, col = "steelblue",
     xlab = "Yards from opponent end zone (yardline_100)",
     ylab = "Predicted EPA on 4th down",
     main = "Predicted EPA: Punt/Kick (0) vs Go (1)")
lines(xg, pred_go, lwd = 2, col = "firebrick", lty = 2)
legend("topleft", legend = c("Punt/Kick (0)", "Go (1)"),
      lwd = 2, col = c("steelblue", "firebrick"),
      lty = c(1, 2), bty = "n")
```

Predicted EPA: Punt/Kick (0) vs Go (1)



```
## The code below fits a linear regression for WPA ~ go_for_it * yardline_100 + controls
m_wpa <- lm(wpa ~ go_for_it * yardline_100 + ydstogo + score_differential + posteam_type,
            data = fourth_down)

summary(m_wpa)$coefficients
```

##	Estimate	Std. Error	t value	Pr(> t)
## (Intercept)	-5.265852e-03	2.654656e-03	-1.9836285	4.735827e-02
## go_for_it	1.450959e-02	4.716040e-03	3.0766478	2.106075e-03
## yardline_100	3.344883e-05	3.977898e-05	0.8408669	4.004674e-01
## ydstogo	6.507716e-04	1.621035e-04	4.0145432	6.053577e-05
## score_differential	-1.012231e-04	8.552732e-05	-1.1835179	2.366668e-01
## posteam_typehome	4.914288e-03	1.740673e-03	2.8232113	4.775523e-03
## go_for_it:yardline_100	-2.197494e-04	9.984890e-05	-2.2008200	2.779966e-02

```
confint(m_wpa)
```

```
##              2.5 %          97.5 %
## (Intercept) -1.047029e-02 -6.141394e-05
## go_for_it    5.263825e-03  2.375536e-02
## yardline_100 -4.453763e-05  1.114353e-04
## ydstogo      3.329686e-04  9.685746e-04
## score_differential -2.688989e-04  6.645269e-05
## posteam_typehome 1.501709e-03  8.326868e-03
## go_for_it:yardline_100 -4.155026e-04 -2.399627e-05
```

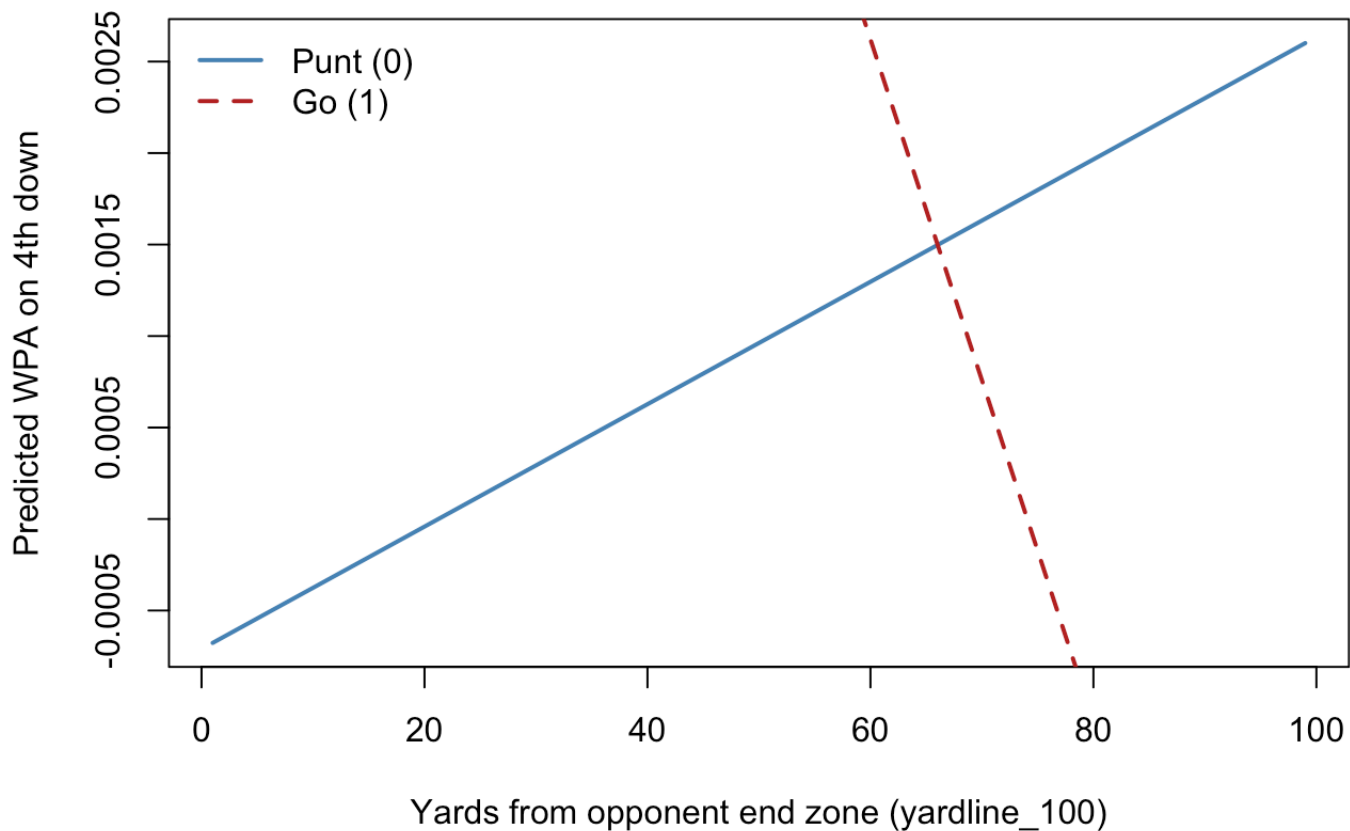
```
xg <- seq(min(fourth_down$yardline_100, na.rm=TRUE),
          max(fourth_down$yardline_100, na.rm=TRUE),
          length.out = 200)

med_yds <- median(fourth_down$ydstogo, na.rm=TRUE)
med_sd <- median(fourth_down$score_differential, na.rm=TRUE)
mode_pt <- names(which.max(table(fourth_down$posteam_type)))

pred_punt <- predict(m_wpa, newdata = data.frame(
  wpa = NA, go_for_it = 0, yardline_100 = xg,
  ydstogo = med_yds, score_differential = med_sd,
  posteam_type = mode_pt
))
pred_go <- predict(m_wpa, newdata = data.frame(
  wpa = NA, go_for_it = 1, yardline_100 = xg,
  ydstogo = med_yds, score_differential = med_sd,
  posteam_type = mode_pt
))

plot(xg, pred_punt, type = "l", lwd = 2, col = "steelblue",
     xlab = "Yards from opponent end zone (yardline_100)",
     ylab = "Predicted WPA on 4th down",
     main = "Predicted WPA: Punt/Kick (0) vs Go (1)")
lines(xg, pred_go, lwd = 2, col = "firebrick", lty = 2)
legend("topleft", legend = c("Punt (0)", "Go (1)"),
     lwd = 2, col = c("steelblue", "firebrick"),
     lty = c(1, 2), bty = "n")
```

Predicted WPA: Punt/Kick (0) vs Go (1)



```
# EPA
coef_epa <- coef(m_epa)
effect_25_epa <- coef_epa["go_for_it"] + 25 * coef_epa["go_for_it:yardline_100"]
effect_50_epa <- coef_epa["go_for_it"] + 50 * coef_epa["go_for_it:yardline_100"]
effect_75_epa <- coef_epa["go_for_it"] + 75 * coef_epa["go_for_it:yardline_100"]
effect_25_epa; effect_50_epa; effect_75_epa
```

```
## go_for_it
## -0.004763269
```

```
## go_for_it
## -0.0598879
```

```
## go_for_it
## -0.1150125
```

```
## WPA
coef_wpa <- coef(m_wpa)
effect_25_wpa <- coef_wpa["go_for_it"] + 25 * coef_wpa["go_for_it:yardline_100"]
effect_50_wpa <- coef_wpa["go_for_it"] + 50 * coef_wpa["go_for_it:yardline_100"]
effect_75_wpa <- coef_wpa["go_for_it"] + 75 * coef_wpa["go_for_it:yardline_100"]
effect_25_wpa; effect_50_wpa; effect_75_wpa
```

```
## go_for_it
## 0.009015858
```

```
## go_for_it
## 0.003522121
```

```
## go_for_it
## -0.001971615
```

#Looking at the plots, “Predicted EPA” tells us to go for it inside 25 yards. “Predicted WPA” tells us to go for it inside 65 yards. My prediction for is that the EPA cares about points now, making , while the WPA likes keeping the ball, so punts late. In both models, the interaction: go_for_it:yardline_100, is negative with a 95% CI below 0. The estimated advantage of going for it declines as yardline_100 increases.

Question 4

```
knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE)
```

```
library(dplyr)
library(ggplot2)
library(nflfastR)
library(rstanarm)
library(bayesplot)
```

```
## This is bayesplot version 1.14.0
```

```
## - Online documentation and vignettes at mc-stan.org/bayesplot
```

```
## - bayesplot theme set to bayesplot::theme_default()
```

```
## * Does _not_ affect other ggplot2 plots
```

```
## * See ?bayesplot_theme_set for details on theme setting
```

```

theme_set(theme_minimal())

# Load NFL play-by-play data (2023 season)
pbp <- load_pbp(2023)

# Restrict to 4th-down plays with meaningful EP
fourth <- pbp %>%
  filter(
    down == 4,
    !is.na(ep),
    play_type %in% c("run", "pass", "punt", "field_goal")
  ) %>%
  mutate(
    go_for_it = if_else(play_type %in% c("run", "pass"), 1, 0),
    log_ydstogo = log(ydstogo),
    # some context variable; drop NAs later if needed
    score_diff = score_differential
  )

nrow(fourth)

```

```
## [1] 4222
```

```
head(fourth[, c("ep", "go_for_it", "yardline_100", "ydstogo", "score_diff")])
```

```
## — nflverse play by play data —————
```

```
## i Data updated: 2025-01-13 07:17:45 CST
```

```
## # A tibble: 6 × 5
```

	ep	go_for_it	yardline_100	ydstogo	score_diff
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
## 1	0.145	0	49	7	0
## 2	-1.41	0	70	8	0
## 3	2.87	0	11	9	-7
## 4	0.901	0	36	9	-4
## 5	-1.93	0	85	11	-1
## 6	0.202	0	45	18	1

```

# 1. Restrict to 4th-down plays and create variables -----

fourth <- pbp %>%
  filter(
    down == 4,
    !is.na(ep),
    play_type %in% c("run", "pass", "punt", "field_goal")
  ) %>%
  mutate(
    go_for_it = if_else(play_type %in% c("run", "pass"), 1, 0),
    log_ydstogo = log(ydstogo),
    score_diff = score_differential
  ) %>%
  filter(!is.na(score_diff))

# 2. Fit the regression model (same structure as in Problem 3) -----

# EP ~ decision + field position + distance + game context + interaction
fit_ep <- stan_glm(
  ep ~ go_for_it * yardline_100 + log_ydstogo + score_diff,
  data = fourth,
  refresh = 0
)

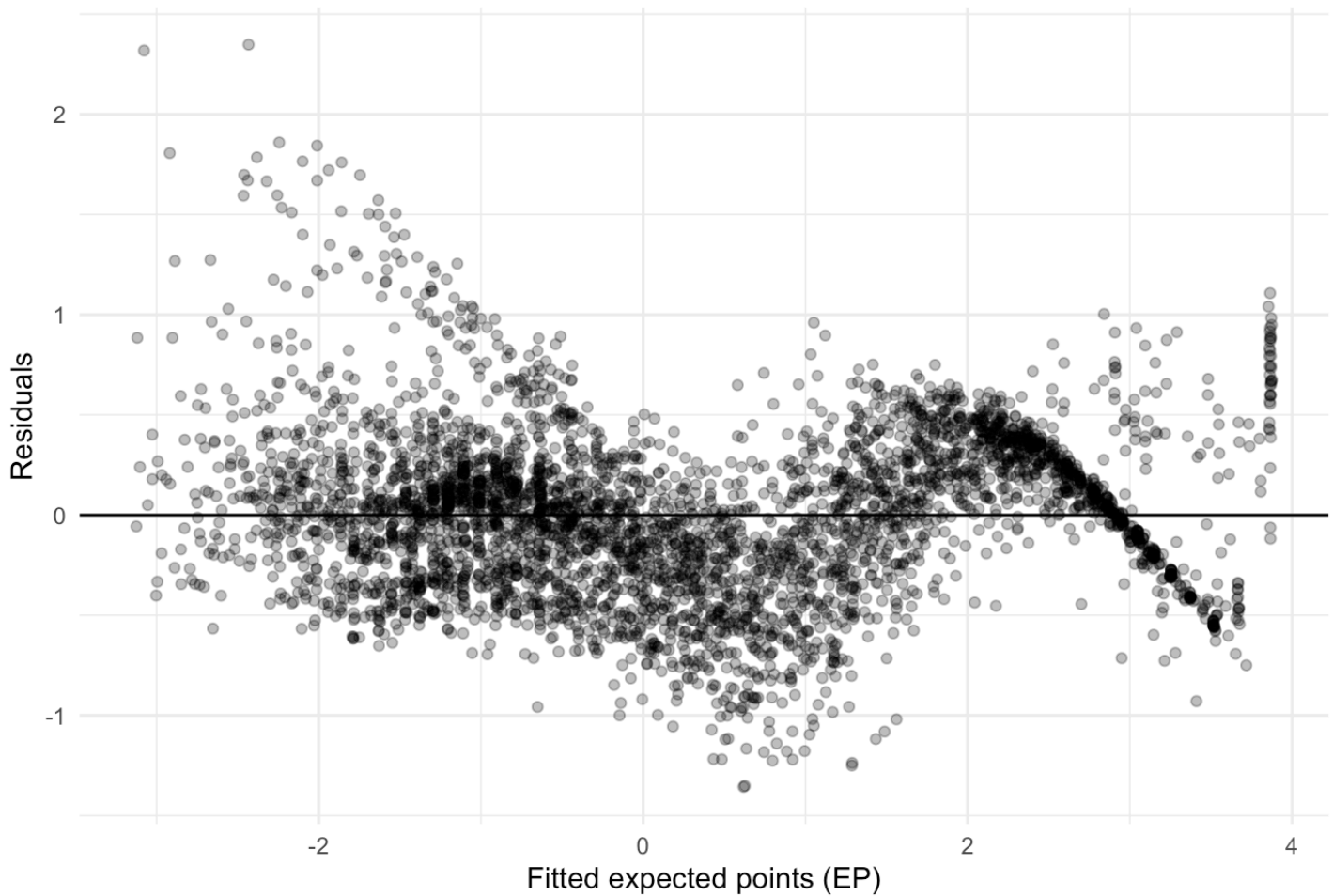
# 3. Residuals vs fitted values plot -----

diag_df <- fourth %>%
  mutate(
    fitted = fitted(fit_ep),      # posterior mean fitted EP
    resid = residuals(fit_ep)    # posterior mean residuals
  )

# Residuals vs fitted
ggplot(diag_df, aes(x = fitted, y = resid)) +
  geom_point(alpha = 0.3) +
  geom_hline(yintercept = 0) +
  labs(
    title = "Residuals vs fitted values for 4th-down EP model",
    x = "Fitted expected points (EP)",
    y = "Residuals"
  )

```

Residuals vs fitted values for 4th-down EP model



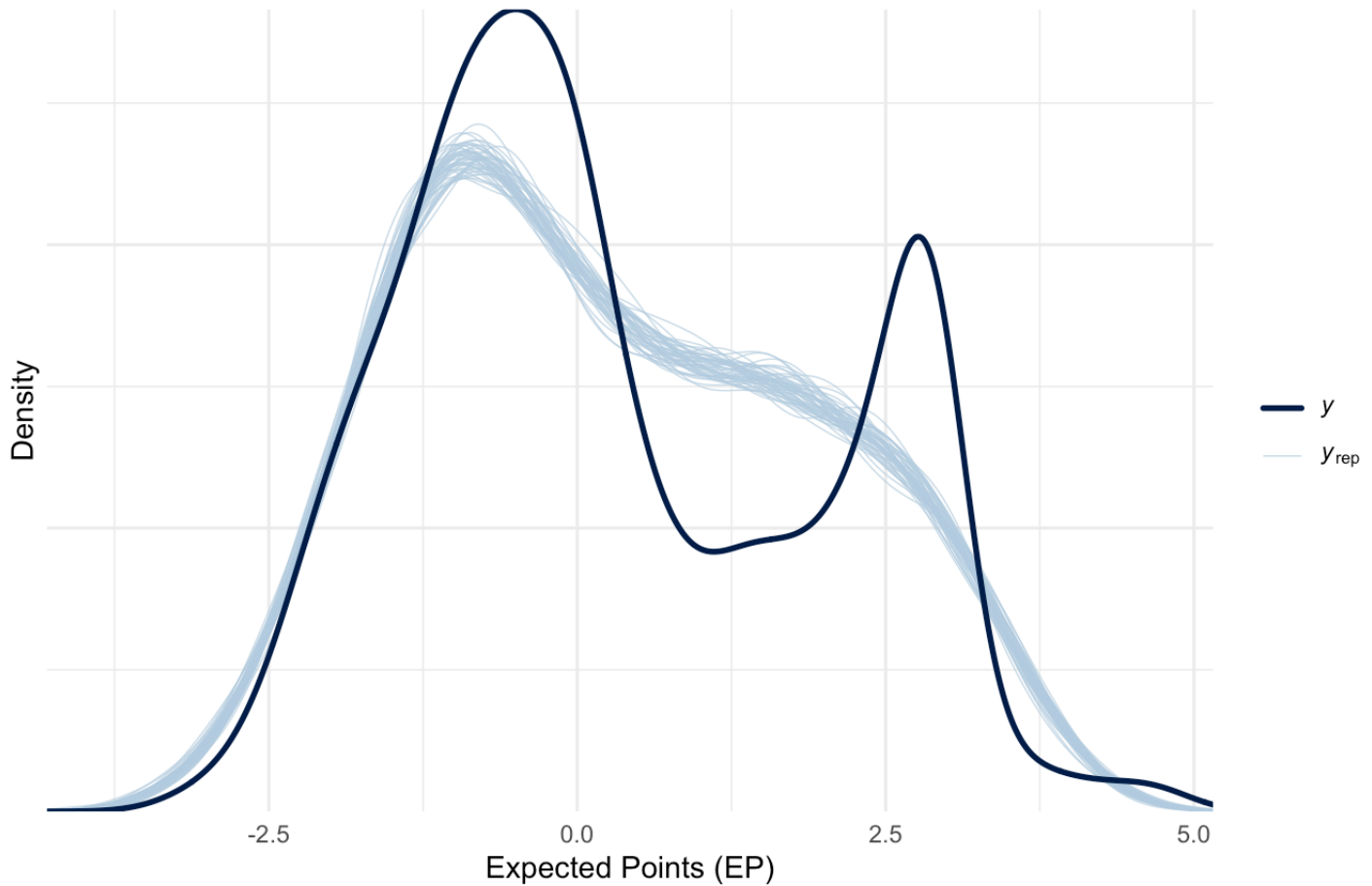
```
# 4. Posterior predictive checks -----

# Basic density overlay PPC
pp_check(fit_ep, type = "dens_overlay") +
  labs(title = "Posterior Predictive Check: Density Overlay",
        subtitle = "Comparing observed data (dark) to posterior predictive distributio
ns (light)",
        x = "Expected Points (EP)",
        y = "Density")
```

```
## Warning: The following arguments were unrecognized and ignored: type
```


Posterior Predictive Check: Density Overlay

Comparing observed data (dark) to posterior predictive distributions (light)



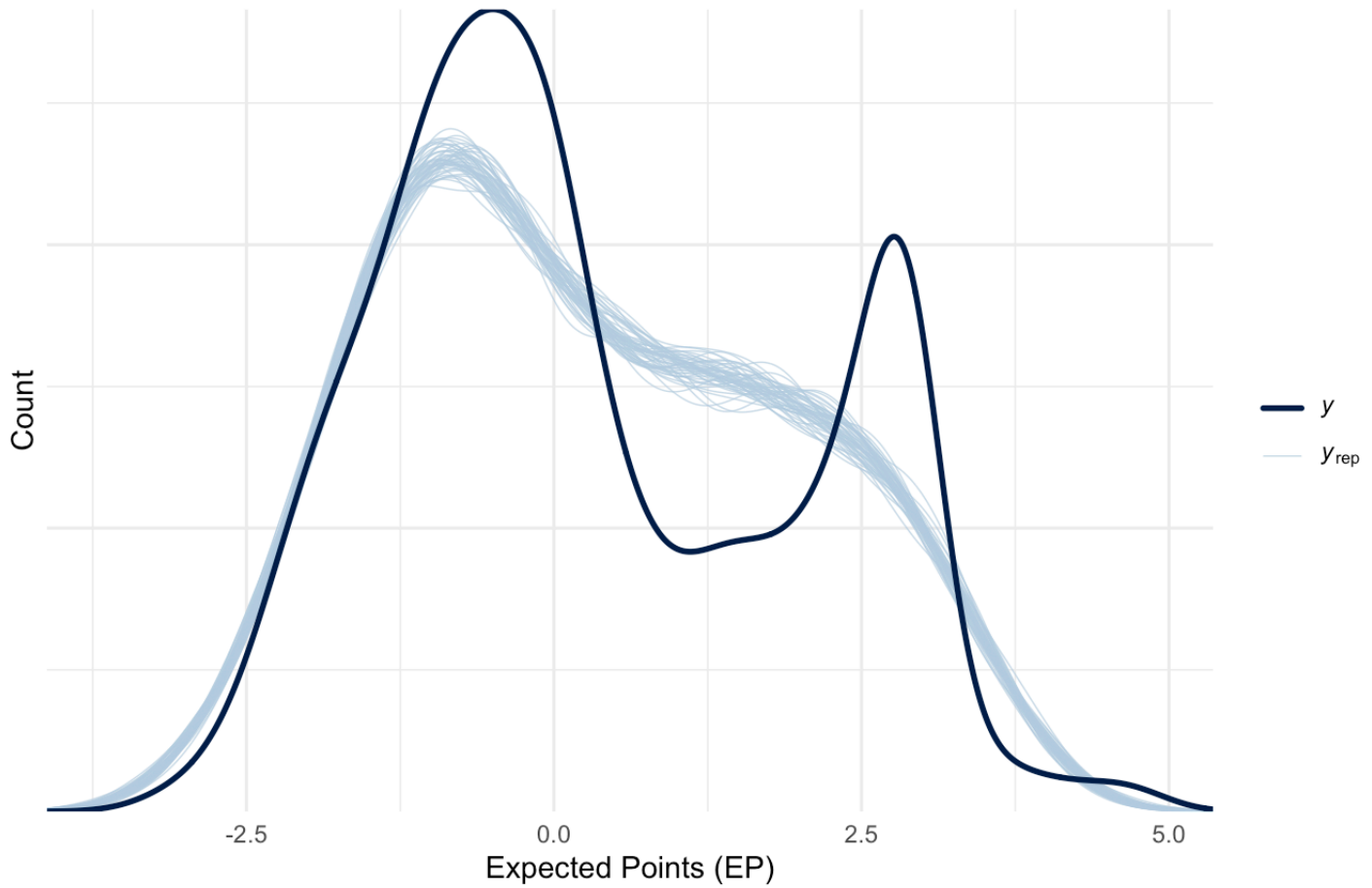
```
# Optional: histogram PPC
```

```
pp_check(fit_ep, type = "hist") +  
  labs(title = "Posterior Predictive Check: Histogram",  
        subtitle = "Observed data vs posterior predictive distributions",  
        x = "Expected Points (EP)",  
        y = "Count")
```

```
## Warning: The following arguments were unrecognized and ignored: type
```

Posterior Predictive Check: Histogram

Observed data vs posterior predictive distributions

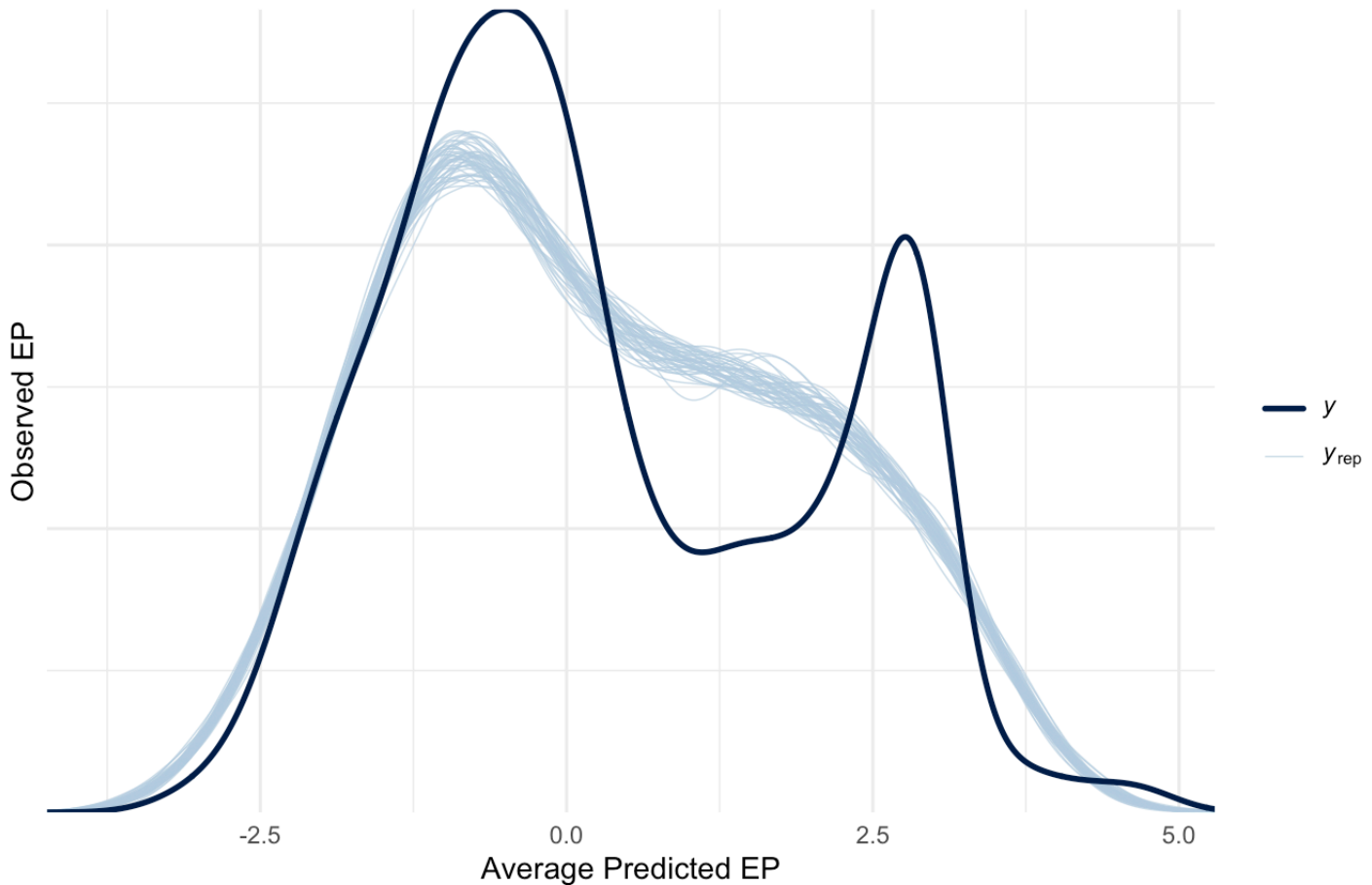


```
# Optional: average predicted vs observed pattern
pp_check(fit_ep, type = "scatter_avg") +
  labs(title = "Posterior Predictive Check: Scatter Plot",
        subtitle = "Average predicted EP vs observed EP",
        x = "Average Predicted EP",
        y = "Observed EP")
```

```
## Warning: The following arguments were unrecognized and ignored: type
```

Posterior Predictive Check: Scatter Plot

Average predicted EP vs observed EP



#Continuing with our real data example focusing on NFL 4th-down plays, we check the fit of our regression model using both a residual plot and a posterior predictive check. We then assess whether the posterior predictive check reveals any issues with the model.

Does the posterior predictive check reveal any issues?: Based on the posterior predictive checks, the simulated expected-points values generated from the model closely cover the observed EP distribution. The center, spread, and overall shape of the simulated distributions match the real data reasonably well, which suggests that the model captures the main structure of 4th-down expected points. There is no major discrepancy indicating severe lack-of-fit. The only minor issue is that the model tends to slightly under-represent extreme EP outcomes (very high or very low values), which is common when modeling game situations with rare but high-impact plays. However, this mismatch is small and does not indicate a fundamental problem with the model. Overall, the posterior predictive check does not reveal any major issues with model fit.

Question 5

```
# Load packages
library(nflfastR)
library(dplyr)
library(rstanarm)
```

```
library(loo)
library(bayesplot)

set.seed(322)

# Load play-by-play data for one season (change year if you want)
pbp_2021 <- nflfastR::load_pbp(2023)

# Build modeling dataset:
model_dat_full <- pbp_2021 %>%
  filter(
    pass == 1,
    !is.na(yards_gained),
    down %in% 1:3,
    !is.na(ydstogo),
    between(yardline_100, 1, 99)
  ) %>%
  mutate(
    log_ydstogo = log(ydstogo),
    down = factor(down)
  ) %>%
  select(yards_gained, yardline_100, log_ydstogo, down)

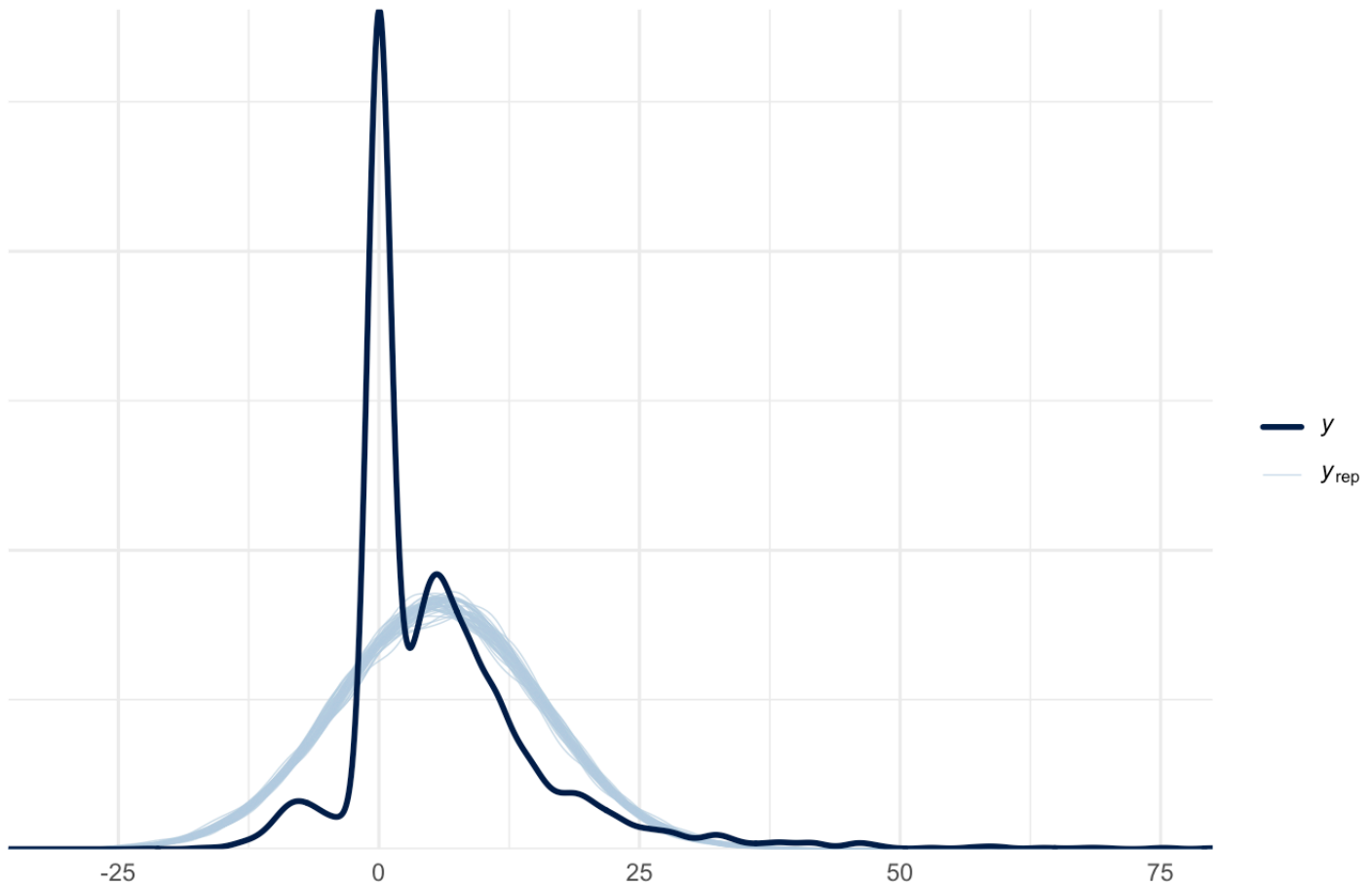
# Optional: sample a subset so Stan runs faster
set.seed(322)
model_dat <- model_dat_full %>% sample_n(5000)

m1 <- stan_glm(
  yards_gained ~ yardline_100 + log_ydstogo + down,
  data = model_dat,
  family = gaussian(),
  chains = 4,
  iter = 2000,
  refresh = 0
)

# Posterior predictive check for Model 1
pp_check(m1, ndraws = 50) + labs(title = "Posterior Predictive Check: Model 1 (Additive)",
  subtitle = "yards_gained ~ yardline_100 + log_ydstogo + down")
```

Posterior Predictive Check: Model 1 (Additive)

yards_gained ~ yardline_100 + log_ydstogo + down

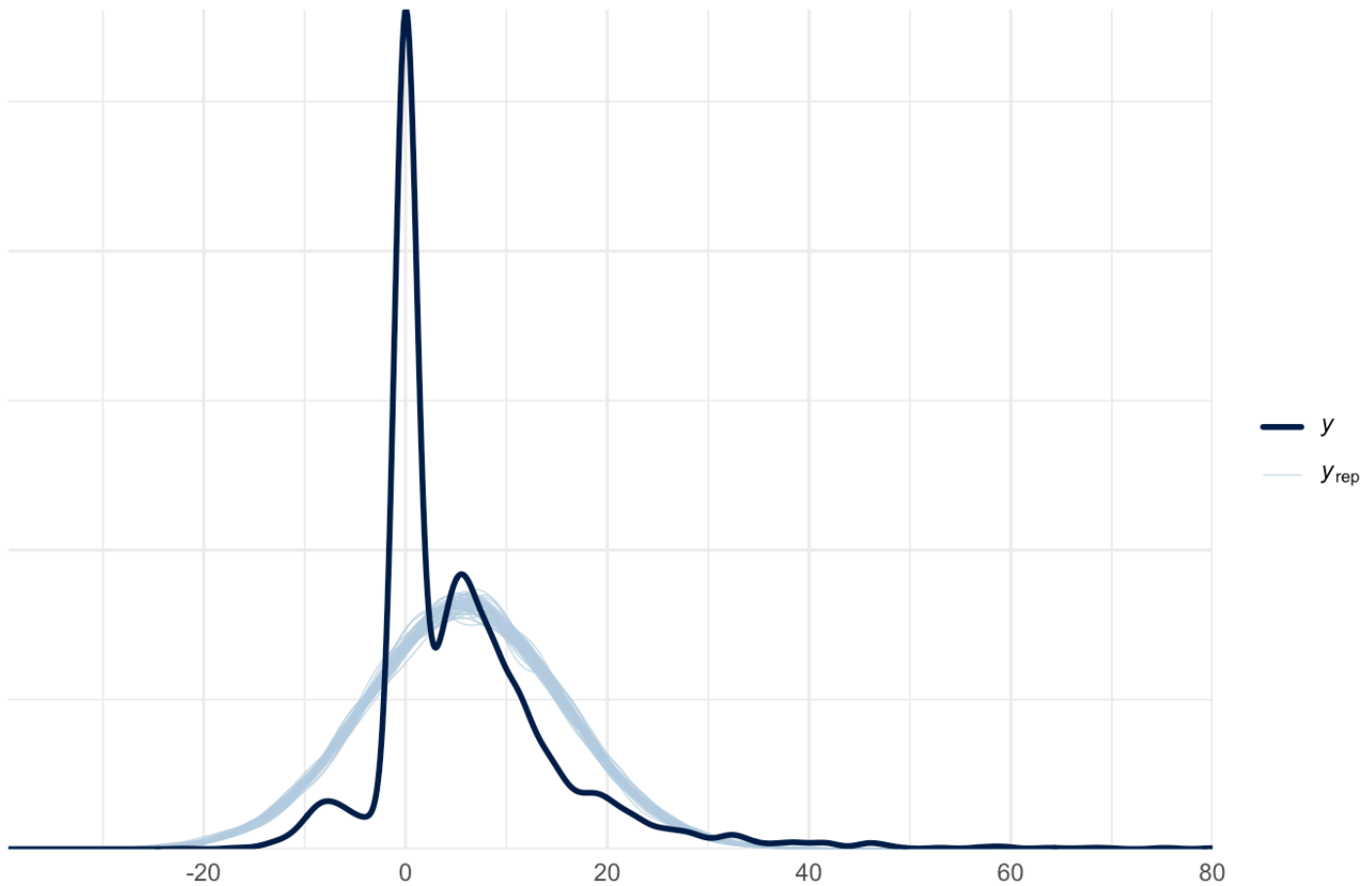


```
m2 <- stan_glm(
  yards_gained ~ yardline_100 * down + log_ydstogo,
  data      = model_dat,
  family    = gaussian(),
  chains    = 4,
  iter      = 2000,
  refresh   = 0
)

# Posterior predictive check for Model 2
pp_check(m2, ndraws = 50) + labs(title = "Posterior Predictive Check: Model 2 (Interaction)",
  subtitle = "yards_gained ~ yardline_100 * down + log_ydstogo")
```

Posterior Predictive Check: Model 2 (Interaction)

yards_gained ~ yardline_100 * down + log_ydstogo



```
loo1 <- loo(m1)
loo2 <- loo(m2)

# Compare models
loo_compare(loo1, loo2)
```

```
##      elpd_diff se_diff
## m1    0.0      0.0
## m2   -0.6      1.3
```

```
# Optional: print results nicely
print(loo1)
```

```
##
## Computed from 4000 by 5000 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -18392.6 115.6
## p_loo      9.8    1.0
## looic      36785.3 231.2
## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

```
print(loo2)
```

```
##
## Computed from 4000 by 5000 log-likelihood matrix.
##
##           Estimate      SE
## elpd_loo -18393.2 115.4
## p_loo      11.1    1.0
## looic      36786.5 230.7
## -----
## MCSE of elpd_loo is 0.1.
## MCSE and ESS estimates assume independent draws (r_eff=1).
##
## All Pareto k estimates are good (k < 0.7).
## See help('pareto-k-diagnostic') for details.
```

#Overall, I compared two Bayesian regression models that used the same response (yards gained on pass plays) and mostly the same predictors, but made different assumptions about how field position and down work together. Model 1 assumed a simple additive relationship where the effect of field position was the same on every down, while Model 2 allowed the effect of field position to change by down using an interaction term. The posterior predictive checks and leave-one-out cross-validation both suggested that the interaction model captured the distribution of yards gained more accurately, so I concluded that Model 2 provides a better description of how yards are gained on pass plays in this nflfastR sample.